Java Encapsulation — 15 Codes

Beginner → Intermediate | Code + Output + Concept

Core idea: Encapsulation means keeping data and methods together in a class and restricting direct access to data, commonly using **private** fields and controlled methods.

1. Basic Getters and Setters

Private data is accessed through public getter and setter methods.

```
class Student {
    private String name;

    public void setName(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.setName("Pratham");
        System.out.println("Name: " + s.getName());
    }
}
```

```
Output:
Name: Pratham
```

2. Encapsulating Age

The age variable is protected using private.

```
class Person {
    private int age;

    public void setAge(int age) {
        this.age = age;
    }

    public int getAge() {
        return age;
    }
}

public class Main {
    public static void main(String[] args) {
        Person p = new Person();
        p.setAge(22);
        System.out.println("Age: " + p.getAge());
    }
}
```

```
Output:
Age: 22
```

3. Multiple Private Variables

Several fields can be encapsulated in one class.

```
class Student {
    private String name;
    private int age;
    private double marks;

    public void setName(String name) { this.name = name; }
    public void setAge(int age) { this.age = age; }
    public void setMarks(double marks) { this.marks = marks; }

    public String getName() { return name; }
    public int getAge() { return age; }
```

```
    public double getMarks() { return marks; }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.setName("Rahul");
        s.setAge(21);
        s.setMarks(85.5);

        System.out.println("Name: " + s.getName());
        System.out.println("Age: " + s.getAge());
        System.out.println("Marks: " + s.getMarks());
    }
}
```

```
Output:
Name: Rahul
Age: 21
Marks: 85.5
```

4. Encapsulation with Constructor

The constructor initializes private fields; getters provide access.

```
class Employee {
    private String name;
    private int salary;

    Employee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }

    public String getName() { return name; }
    public int getSalary() { return salary; }
}

public class Main {
    public static void main(String[] args) {
        Employee e = new Employee("Amit", 30000);

        System.out.println("Name: " + e.getName());
        System.out.println("Salary: " + e.getSalary());
    }
}
```

```
Output:
Name: Amit
Salary: 30000
```

5. Setter Validation

A setter can validate data before storing it.

```
class BankAccount {
    private double balance;

    public void setBalance(double balance) {
        if (balance >= 0) {
            this.balance = balance;
        } else {
            System.out.println("Invalid balance");
        }
    }

    public double getBalance() {
        return balance;
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount a = new BankAccount();
        a.setBalance(5000);
        System.out.println("Balance: " + a.getBalance());
        a.setBalance(-100);
    }
}
```

```
Output:
Balance: 5000.0
Invalid balance
```

6. Read-Only Encapsulation

There is a getter but no setter, so outside code cannot change price.

```
class Product {
    private int price = 1500;

    public int getPrice() {
        return price;
    }
}

public class Main {
    public static void main(String[] args) {
        Product p = new Product();
        System.out.println("Price: " + p.getPrice());
    }
}
```

```
Output:
Price: 1500
```

7. Write-Only Style

The actual password is not exposed through a getter.

```
class Password {
    private String password;

    public void setPassword(String password) {
        this.password = password;
    }

    public void showPasswordLength() {
        System.out.println("Password length: " + password.length());
    }
}

public class Main {
    public static void main(String[] args) {
        Password p = new Password();
        p.setPassword("java123");
        p.showPasswordLength();
    }
}
```

```
Output:
Password length: 7
```

8. Deposit Method

Controlled methods can change private data instead of direct access.

```
class BankAccount {
    private double balance;

    public void deposit(double amount) {
        if (amount > 0) {
            balance = balance + amount;
        }
    }

    public double getBalance() {
        return balance;
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount a = new BankAccount();
        a.deposit(1000);
        a.deposit(500);
        System.out.println("Balance: " + a.getBalance());
    }
}
```

```
Output:
Balance: 1500.0
```

9. Withdraw Validation

The class controls how balance can be changed.

```
class BankAccount {
    private double balance = 5000;

    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance = balance - amount;
            System.out.println("Withdrawal successful");
        } else {
            System.out.println("Invalid withdrawal");
        }
    }

    public double getBalance() {
        return balance;
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount a = new BankAccount();
        a.withdraw(2000);
        a.withdraw(5000);
        System.out.println("Final Balance: " + a.getBalance());
    }
}
```

```
Output:
Withdrawal successful
Invalid withdrawal
Final Balance: 3000.0
```

10. Boolean Encapsulation

Boolean getters commonly use the isFieldName() pattern.

```
class User {
    private boolean loggedIn;

    public void setLoggedIn(boolean loggedIn) {
        this.loggedIn = loggedIn;
    }

    public boolean isLoggedIn() {
        return loggedIn;
    }
}

public class Main {
    public static void main(String[] args) {
        User u = new User();
        u.setLoggedIn(true);
        System.out.println("Logged in: " + u.isLoggedIn());
    }
}
```

```
Output:
Logged in: true
```

11. Encapsulation with Calculation

Private data can be used internally while exposing useful operations.

```
class Rectangle {
    private double length;
    private double width;

    public void setLength(double length) { this.length = length; }
    public void setWidth(double width) { this.width = width; }

    public double getArea() {
        return length * width;
    }
}

public class Main {
    public static void main(String[] args) {
        Rectangle r = new Rectangle();
        r.setLength(10);
        r.setWidth(5);
        System.out.println("Area: " + r.getArea());
    }
}
```

```
Output:
Area: 50.0
```

12. Private ID + Constructor

Private identity data is initialized through a constructor.

```
class Employee {
    private int id;
    private String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public int getId() { return id; }
    public String getName() { return name; }
}

public class Main {
    public static void main(String[] args) {
        Employee e = new Employee(101, "Neha");
        System.out.println("ID: " + e.getId());
        System.out.println("Name: " + e.getName());
    }
}
```

```
Output:
ID: 101
Name: Neha
```

13. Controlled Marks Update

The setter prevents invalid values from changing object state.

```
class Student {
    private int marks;

    public void setMarks(int marks) {
        if (marks >= 0 && marks <= 100) {
            this.marks = marks;
        } else {
            System.out.println("Marks must be between 0 and 100");
        }
    }

    public int getMarks() {
        return marks;
    }
}

public class Main {
```

```
    public static void main(String[] args) {
        Student s = new Student();
        s.setMarks(80);
        System.out.println("Marks: " + s.getMarks());

        s.setMarks(120);
        System.out.println("Marks: " + s.getMarks());
    }
}
```

```
Output:
Marks: 80
Marks must be between 0 and 100
Marks: 80
```

14. Controlled Speed Methods

Outside code cannot set an arbitrary speed; it must use controlled methods.

```
class Fan {
    private int speed;

    public void increaseSpeed() {
        if (speed < 5) speed++;
    }

    public void decreaseSpeed() {
        if (speed > 0) speed--;
    }

    public int getSpeed() {
        return speed;
    }
}

public class Main {
    public static void main(String[] args) {
        Fan f = new Fan();

        f.increaseSpeed();
        f.increaseSpeed();
        f.increaseSpeed();

        System.out.println("Speed: " + f.getSpeed());

        f.decreaseSpeed();
        System.out.println("Speed: " + f.getSpeed());
    }
}
```

```
Output:
Speed: 3
Speed: 2
```

15. Intermediate Employee Encapsulation

Combines private fields, constructor, getters, controlled update, validation, and a business method.

```
class Employee {
    private String name;
    private double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public String getName() { return name; }
    public double getSalary() { return salary; }

    public void increaseSalary(double percentage) {
        if (percentage > 0) {
            salary = salary + (salary * percentage / 100);
        }
    }

    public void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("Salary: " + salary);
```

```
    }
}

public class Main {
    public static void main(String[] args) {
        Employee e = new Employee("Rohan", 40000);

        e.displayDetails();

        e.increaseSalary(10);

        System.out.println("After 10% increment:");
        e.displayDetails();
    }
}
```

```
Output:
Name: Rohan
Salary: 40000.0
After 10% increment:
Name: Rohan
Salary: 44000.0
```

Quick Revision

private → hides data
getter → reads private data
setter → changes private data
validation → rejects invalid data
controlled methods → decide how object state changes

Interview definition: Encapsulation is the process of binding data and methods into a single class and restricting direct access to the data using access modifiers such as private.